

I. WHAT IS PYTORCH?

PyTorch is an open-source machine learning library primarily developed by Meta's AI Research lab. It is preferred by researchers and students because of its **Dynamic Computational Graph**, which allows you to change how the network behaves while it is running.

1.1. PyTorch vs. TensorFlow

- **PyTorch:** "Pythonic" and flexible. It feels like native Python code.
- **TensorFlow:** More "Static" and often used for massive industrial deployments, though PyTorch is catching up in production with TorchScript.

II. THE ATOM OF AI: THE TENSOR

A **Tensor** is the fundamental data structure in PyTorch. It is essentially a multi-dimensional array, similar to NumPy's ndarray, but with two superpower features:

1. **GPU Acceleration:** Tensors can be moved to an NVIDIA GPU (using CUDA) to speed up math by 50x–100x.
2. **Automatic Differentiation:** Tensors keep track of every operation performed on them to calculate gradients automatically.

2.1. Tensor Operations

- **Reshaping:** Changing a 28 \times 28 image matrix into a flat vector of 784 pixels using `.view()` or `.reshape()`.
- **Matrix Multiplication:** Performed using the `@` operator or `torch.matmul()`. This is how layers pass data to each other.

III. THE "AUTOGGRAD" ENGINE

In File 16, we discussed **Backpropagation**. In the old days, scientists had to derive complex calculus formulas by hand. PyTorch uses a module called **Autograd** to do this automatically.

3.1. How Autograd Works

When you create a tensor with `requires_grad=True`, PyTorch builds a graph of every operation. When you call `.backward()`, it automatically calculates the derivative (gradient) of the loss with respect to every weight in the network.

IV. BUILDING BLOCKS OF A PYTORCH MODEL

Every model in PyTorch follows a specific class structure inherited from `nn.Module`.

1. **`__init__` method:** Where you define your layers (Linear, Convolutional, etc.).
2. **`forward` method:** Where you define how data flows through the layers.
3. **The Optimizer:** (e.g., SGD, Adam) The algorithm that updates the weights based on the gradients.
4. **The Criterion (Loss Function):** (e.g., `nn.CrossEntropyLoss`) How the model measures its error.

V. PRACTICAL LAB: THE TRAINING LOOP

This is the standard code "Skeleton" for training any AI model in PyTorch.

Python

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
# 1. Setup Device (Use GPU if available)
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
# 2. Define Data and Model
```

```
X = torch.randn(100, 10).to(device) # 100 samples, 10 features
```

```
y = torch.randint(0, 2, (100,)).to(device) # Binary labels
```

```
model = nn.Sequential(
```

```
    nn.Linear(10, 5),
```

```
    nn.ReLU(),
```

```
    nn.Linear(5, 2)
```

```
).to(device)
```

```
# 3. Setup Loss and Optimizer
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

```
# 4. THE TRAINING LOOP
```

```
for epoch in range(10):
```

```
    # Forward Pass
```

```
    outputs = model(X)
```

```
    loss = criterion(outputs, y)
```

```
# Backward Pass (The magic part)

optimizer.zero_grad() # Clear old gradients

loss.backward()      # Calculate new gradients

optimizer.step()     # Update weights


print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

VI. PERFORMANCE OPTIMIZATION: DATA LOADERS

When working with Big Data, you cannot load 1 Terabyte of images into RAM at once.

- **Dataset class:** Defines how to access a single data point.
- **DataLoader class:** Handles "Batching" (loading 32 or 64 items at a time), shuffling, and multi-processing to keep the GPU busy.